

The complete guide to Retrieval Augmented Generation (RAG) in Azure Confidential Virtual Machines

Deploy confidential RAG in Azure with hardware acceleration (AMD 4th Gen EPYC processors, Intel® TDX on 4th Gen Intel® Xeon Scalable Processors and NVIDIA H100 Tensor Core GPUs)



Contents

Introduction	3
Objective	4
RAG reference solution	5
Advanced RAG and GenAI reference solution with open source	6
Reference solution with Charmed OpenSearch and KServe	7
RAG security and confidentiality	11
Confidential AI	11
Confidential VM in Azure	13
Deployment guide	15
Prerequisites	15
Install Juju and configure Azure credentials	15
Bootstrap Juju controller and create Juju model for Charmed OpenSearch	17
Deploy Charmed OpenSearch and set up the RAG services	17
Ask and start conversational flow with your RAG	24
Conclusion	25
Confidential RAG in Canonical	26

Introduction

One of the most critical gaps in traditional Large Language Models (LLMs) is that they rely on static knowledge already contained within them. Basically, they might be very good at understanding and responding to prompts, but they often fall short in providing the latest or highly specific information. This is where Retrieval Augmentation Generation (RAG) comes in; RAG addresses these critical gaps in traditional LLMs by incorporating current and new information that serves as a reliable source of truth for these models.

RAG enhances generative AI models by utilizing external knowledge sources such as documents and databases. These external knowledge bases provide verifiable, up-to-date information that can improve machine learning (ML) models by reducing errors, simplifying implementation, and lowering the cost of continuous re-training. To learn more about RAG and learn what it is in more depth, you can read [our blog covering this technique](#).

Building a [robust generative AI infrastructure](#), such as those for RAG, can be complex and challenging. It requires careful consideration of the technology stack, data, scalability, ethics, and security. For the technology stack, the hardware, operating systems, cloud services, and generative AI services must be resilient and efficient based on the scale that enterprises require. Given the concerns that enterprises have regarding the cost, vendor lock-in, data sovereignty, regulatory compliance, and complexity of AI projects, open source software represents a powerful way to accomplish project goals while maintaining full control over sensitive data and avoiding restrictions and heavy overheads. There are several to choose from on the market, but in this white paper we will focus on Charmed OpenSearch and KServe.

When implementing a generative AI use case and developing a robust infrastructure, it is crucial to prioritize the security and confidentiality of both data and machine learning models. These assets are invaluable, and protecting them is essential for maintaining trust, ensuring legal compliance, supporting business continuity, and preventing malicious attacks. RAG can be particularly vulnerable in this regard, which is especially concerning for mission-critical applications in sectors like financial services and public services.

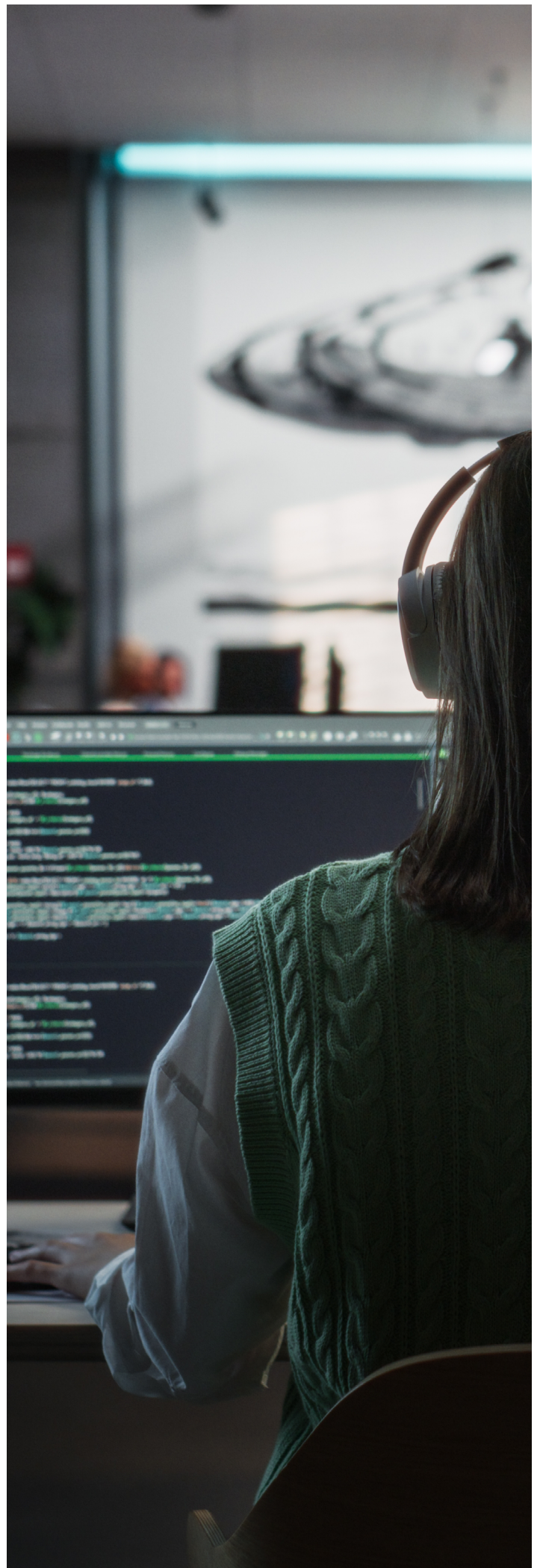
Of course, enterprises also have concerns about the risks of performing machine learning tasks in the cloud. For example, they may worry about the potential compromise of their sensitive data or their intellectual property. In some cases, there are regulatory challenges to consider, such as stringent industry regulations that may prohibit the sharing of sensitive data. This makes it difficult, or even outright impossible, to utilize large amounts of valuable private data, which can heavily limit the true potential of AI across crucial domains. Confidential AI tackles this problem directly, providing a hardware-rooted execution environment that spans both the Central Processing Unit (CPU) and Graphics Processing Unit (GPU). This environment enhances the protection of AI data and code at runtime by helping to safeguard it against privileged system software (such as the hypervisor or host OS) and privileged operators in the cloud.

Objective

This guide focuses on creating a RAG workflow using open source tools. In addition, the guide emphasizes the importance of security and data confidentiality, while also presenting security-related innovations such as confidential computing, that can help secure your RAG setup.

It is intended for data enthusiasts, engineers, scientists, and machine learning professionals who want to develop RAG solutions on public cloud platforms like Azure, utilizing enterprise open source tools that are not part of Azure's native microservices offering. This guide can support various projects, including proofs of concept, development, and production.

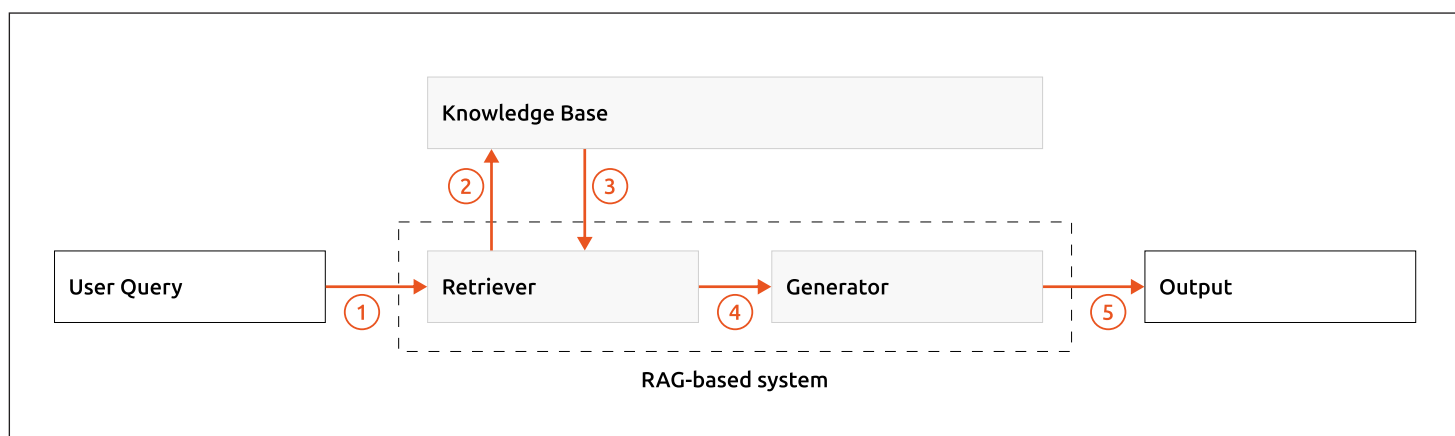
Please note that while this guide highlights specific [open source tools](#), other tools not mentioned can be used as alternatives. If you decide to use different tools, be sure to adjust the hardware specifications – such as storage capacity, computing power, and configuration – to meet the specific requirements of your use case.



RAG reference solution

When a query is made in an AI chatbot, the RAG-based system first retrieves relevant information from a large dataset or knowledge base, and then uses this information to inform and guide the generation of the response. The RAG-based system consists of two key components. The first component is the Retriever, which is responsible for locating relevant pieces of information that can help answer a user query. It searches a database to select the most pertinent information. This information is then provided to the second component, the Generator. The Generator is a large language model that produces the final output.

Before using your RAG-based system, you must first create your knowledge base, which consists of external data that is not included in your LLM training data. This external data can originate from various sources, including documents, databases, and API calls. Most RAG systems utilize an AI technique called model embedding, which converts data into numerical representations and stores it in a [vector database](#). By using an embedding model, you can create a knowledge model that is easily understandable and readily retrievable in the context of AI. Once you have a knowledge base and a vector database set up, you can now perform your RAG process; here is a conceptual flow:



This conceptual flow follows 5 general steps:

1. The user enters a prompt or query.
2. Your Retriever searches for relevant information from a knowledge base. The relevance can be determined using mathematical vector calculations and representations through a vector search and database functionality.
3. The relevant information is retrieved to provide enhanced context to the generator.
4. The query and prompts are now enriched with this context and are ready to be augmented for use with a large language model using prompt engineering techniques. The augmented prompt enables the language model to respond accurately to your query.
5. Finally, the generated text response is delivered to you.

Advanced RAG and GenAI reference solution with open source

RAG can be used in various applications, such as AI chatbots, semantic search, data summarization, and even code generation. The reference solution below outlines how RAG can be combined with advanced generative AI reference architectures to create optimized LLM projects that provide contextual solutions to various GenAI use cases.

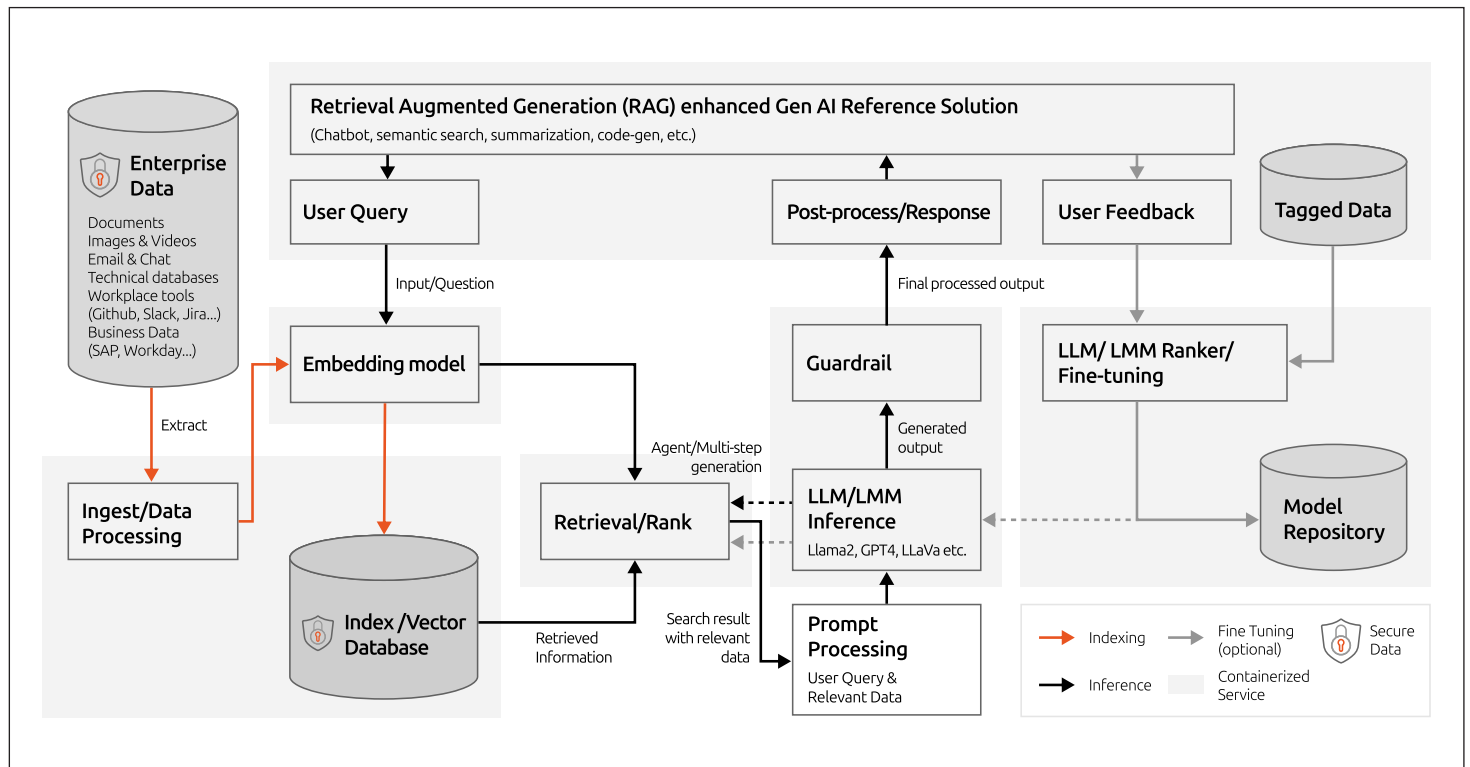


Figure 1: RAG enhanced GenAI Ref solution (source: <https://opea.dev/>)

The GenAI blueprint mentioned above was published by **OPEA (Open Platform for Enterprise AI)**, a project of the Linux Foundation. The aim of creating this blueprint is to establish a framework of composable building blocks for state-of-the-art generative AI systems, which include LLM, data storage, and prompt engines. Additionally, it provides a blueprint for RAG and outlines end-to-end workflows. The **recent release 1.1** of the Opea project showcased multiple GenAI projects that demonstrate how RAG systems can be enhanced through open source tools.

Each service within the blocks has distinct tasks to perform, and there are various open source solutions available that can help to accelerate these services based on enterprise needs.

Reference solution with Charmed OpenSearch and KServe

When building a generative AI project, such as a RAG system and advanced generative AI reference architectures, it is crucial to include multiple components and services. These components typically encompass databases, knowledge bases, retrieval systems, vector databases, model embeddings, large language models (LLMs), inference engines, prompt processing, and guardrail and fine-tuning services, among others.

RAG allows users to choose the most suitable RAG services and applications for their specific use cases. The reference workflow outlined below mainly utilizes two open source tools: Charmed OpenSearch and KServe. In the RAG workflow depicted below, fine-tuning is not mandatory; however, it can enhance the performance of LLMs as the project scales.

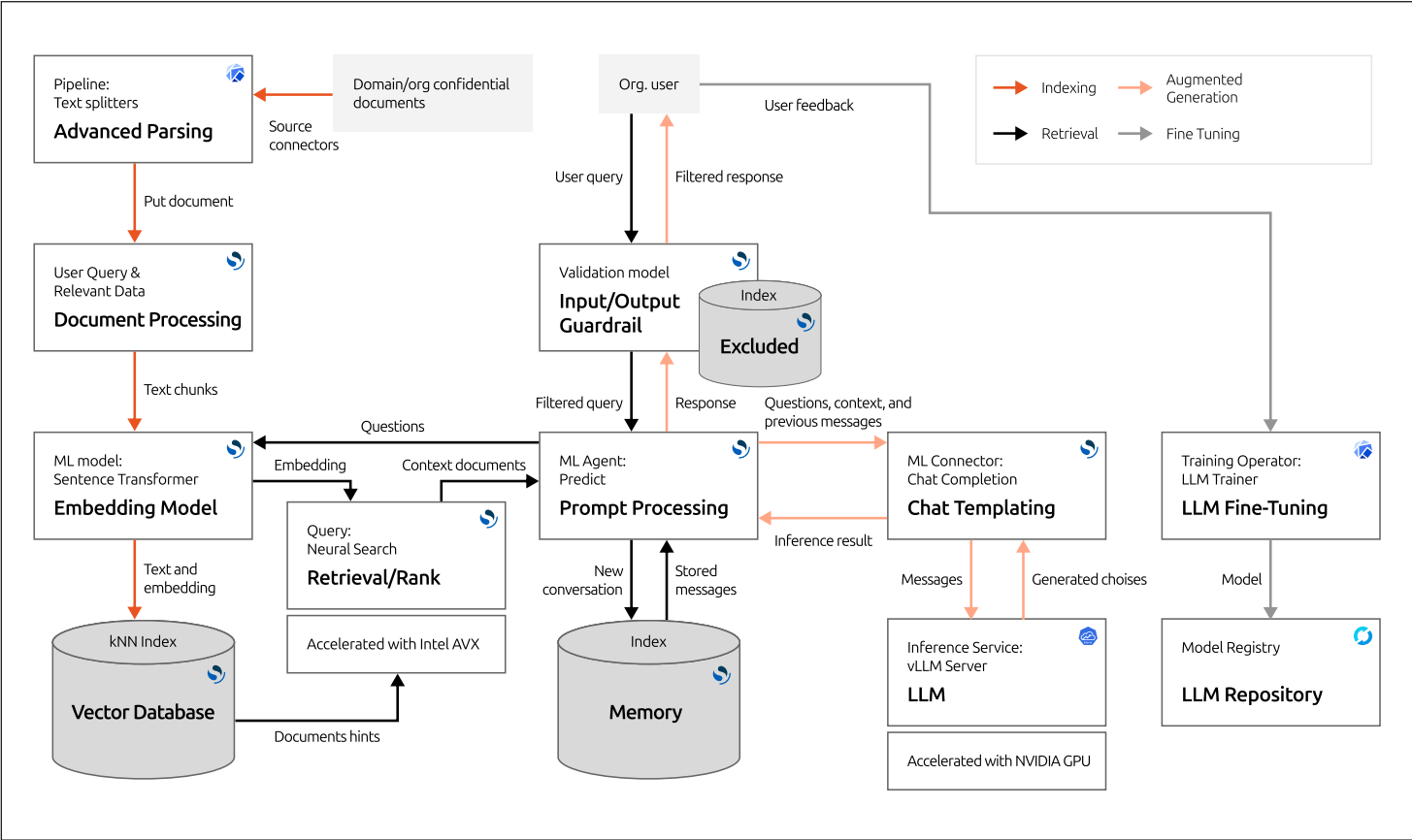


Figure 2: RAG workflow diagram using open source tools

The table below describes all the RAG services highlighted in the workflow diagram above and maps the open source solutions that are used in this guide.

Services	Description	Open source solutions
Advanced parsing	Text splitters are advanced parsing techniques for the document that goes to the RAG system. In this way, the document can be cleaner, and focused and will provide informative input.	Charmed Kubeflow: Text splitters
Ingest/data processing	The ingest or data processing is a data pipeline layer. This is responsible for data extraction, cleansing, and the removal of unnecessary data that you will run.	Charmed OpenSearch can be used for document processing
Embedding model	The embedding model is a machine-learning model that converts raw data to vector representations.	Charmed OpenSearch- Sentence transformer
Retrieval and ranking	This component retrieves the data from the knowledge base; it also ranks the relevance of the information being fetched based on relevance scores.	Charmed OpenSearch with FAISS (Facebook AI Similarity Search)
Vector database	A vector database stores vector embeddings so data can be easily searched by the 'retrieval and ranking services'.	Charmed OpenSearch- KNN Index as a vector database
Prompt processing	This service formats queries and retrieved text into a readable format so it is structured to the LLM.	Charmed OpenSearch - OpenSearch: ML - agent predict
LLM	This component provides the final response using multiple GenAI models.	GPT, Llama, Deepseek
LLM inference	This refers to operationalizing machine learning in production by processing running data into a machine learning model so that it gives an output.	Charmed Kubeflow with Kserve
Guardrail	This component ensures ethical content in the GenAI response by creating a guardrail filter for the inputs and outputs.	Charmed OpenSearch: guardrail validation model
LLM Fine-tuning	Fine-tuning is the process of taking a pre-trained machine learning model and further training it on a smaller, targeted data set.	Charmed Kubeflow
Model repository	This component is used to store and version trained machine learning models, especially within the process of fine-tuning. This registry can track the model's lifecycle from deployment to retirement.	Charmed Kubeflow, Charmed MLFlow
Framework for building LLM application	This simplifies LLM workflow, prompts, and services so that building LLMs is easier.	Langchain

This table provides an overview of the key components involved in building a RAG system and advanced GenAI reference solution, along with associated open source solutions for each service. Each service performs a specific task that can enhance your LLM setup, whether it relates to data management and preparation, embedding a model in your database, or improving the LLM itself.

The deployment guide below will cover most of the services except the following: model repository, LLM fine-tuning, and text splitters.

The rate of innovation in this field, particularly within the open source community, has become exponential. It is crucial to stay updated with the latest developments, including new models and emerging RAG solutions.

**RAG COMPONENT:
CHARMED OPENSEARCH**

Charmed OpenSearch will be mainly used in this RAG workflow deployment. [Charmed OpenSearch](#) is an operator that builds on the [OpenSearch upstream](#) by integrating automation to streamline the deployment, management, and orchestration of production clusters. The operator enhances efficiency, consistency, and security. Its rich [features](#) include high availability, seamless scaling features for deployments of all sizes, HTTP and data-in-transit encryption, multi-cloud support, safe upgrades without downtime, roles and plugin management, and data visualization through Charmed OpenSearch Dashboards.

With the Charmed OpenSearch operator (also known as charm), you can deploy and run OpenSearch on physical and virtual machines (VM) and other cloud and cloud-like environments, including AWS, Azure, Google Cloud, OpenStack, and VMware. For the next section, deployment guide, will be using Azure VM instances:

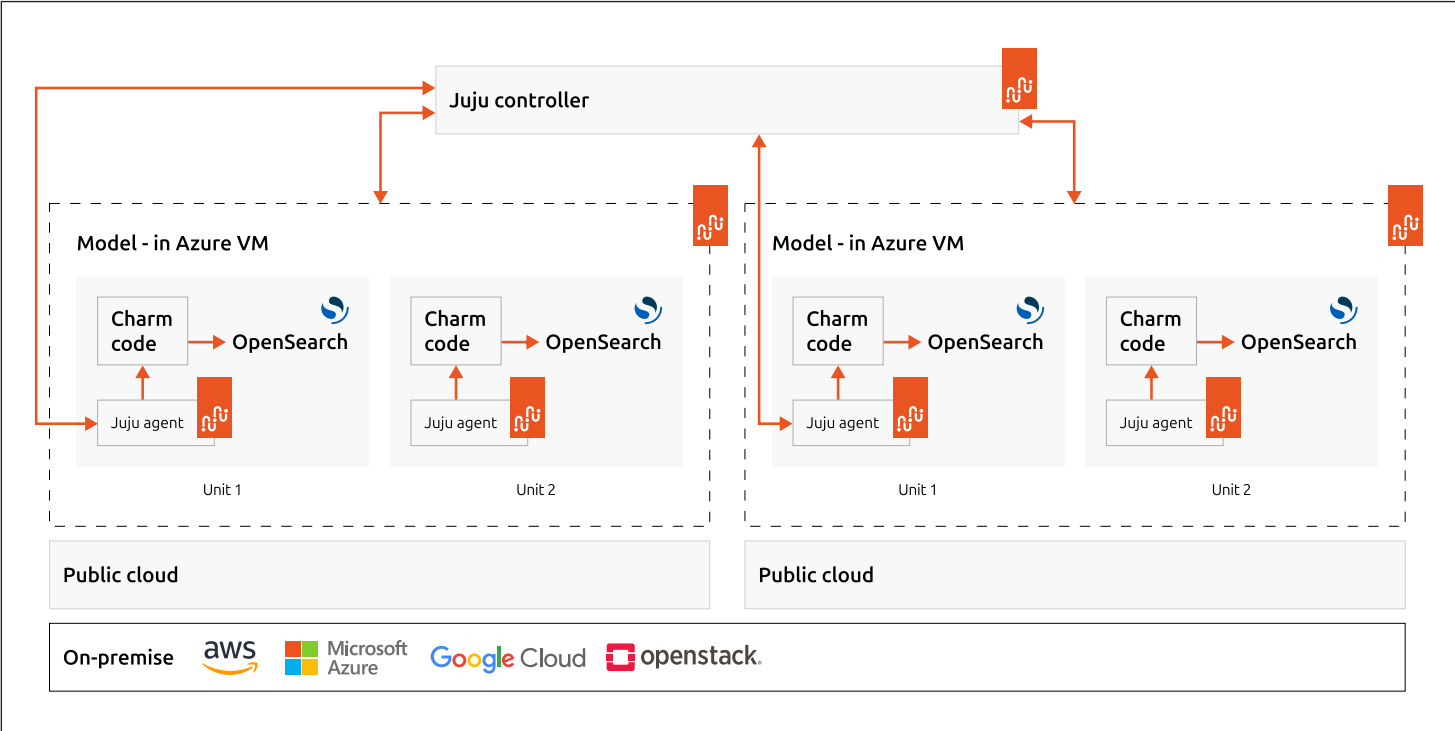


Figure 3: Charmed OpenSearch architecture

Charmed OpenSearch uses [Juju](#). Juju is an open source orchestration engine for software operators that enables the deployment, integration, and lifecycle management of applications at any scale on any infrastructure. In the deployment process, the Juju controller manages the flow of data and interactions within multiple OpenSearch deployments, including mediating between different parts of the system.

Charmed OpenSearch deployment and use is straightforward. If you'd like to learn how to use and deploy it in a range of cloud environments, you can read more in [our in-depth Charmed OpenSearch documentation](#).

RAG COMPONENT: KSERVE

KServe is a cloud-native solution within the Kubeflow ecosystem that serves machine learning models. By leveraging Kubernetes, KServe operates effectively in cloud-native environments. It can be used for various purposes, including model deployment, machine learning model versioning, LLM inference, and model monitoring.

In the RAG use case discussed in this guide, we will use KServe to perform inference on LLM. Specifically, it will process an already-trained LLM to make predictions based on new data. This emphasizes the need for a robust LLM inference system that works with local and public LLMs. The system should be scalable, capable of handling high concurrency, providing low-latency responses, and delivering accurate answers to LLM-related questions.

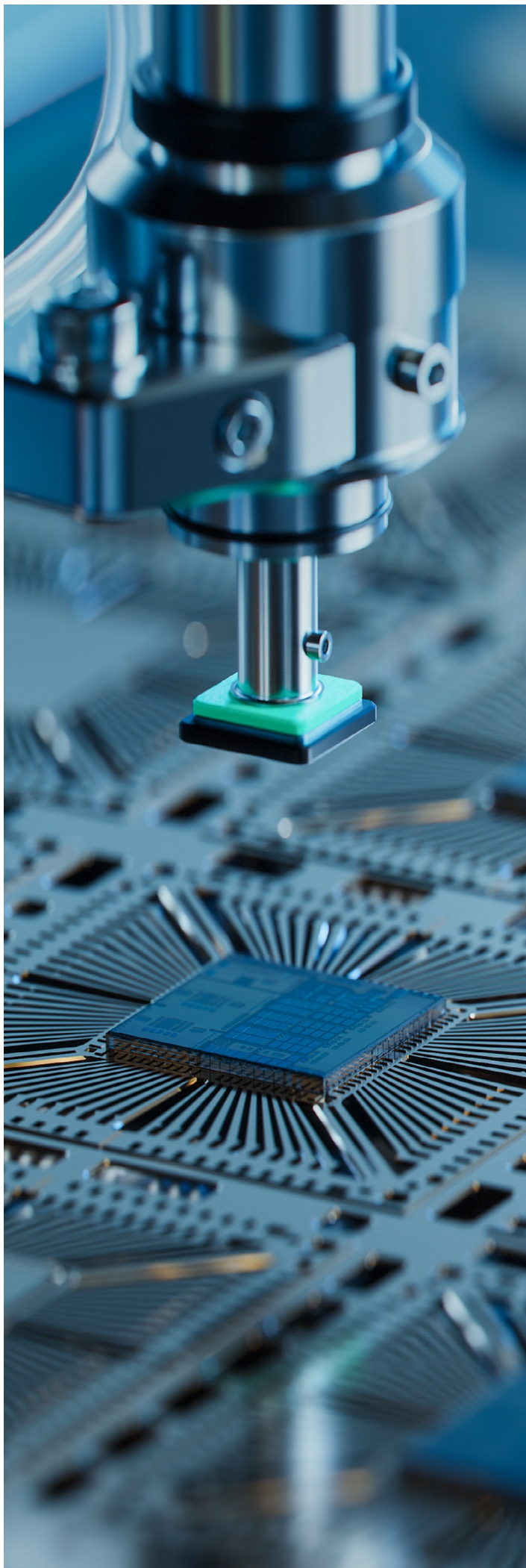
In the deployment guide, we'll take you through a comprehensive and hands-on guide to building and deploying a RAG service using Charmed OpenSearch and KServe. Charmed Kubeflow by Canonical natively supports KServe.

Confidential AI

[Confidential AI](#) is made possible thanks to [confidential computing](#), a new technology that represents a significant departure from the traditional threat model of public clouds. In the past, vulnerabilities within the extensive codebase of the cloud's privileged system software, including the operating system, hypervisor, and firmware, posed a constant risk to the confidentiality and integrity of running code and data. Similarly, unauthorized access by a malicious cloud administrator could compromise the security of your virtual machine (VM) and its platform. This process is mapped in the attestation figure below:

While confidential computing efforts have historically focused primarily on CPUs, the advent of NVIDIA H100 GPUs with confidential computing capabilities opens up new possibilities for extending this security paradigm to GPUs. The Azure solution, which integrates both CPU and GPU components, is what makes confidential AI achievable. At a high level, this solution relies on the following components:

- **CPU-TEE:** Ubuntu confidential VMs that run on the Intel® TDX on 4th Gen Intel® Xeon Scalable Processors are available in Azure. They introduce secure and isolated virtual machines called trust domains (TDs), designed to shield against diverse software threats, including those posed by VM managers and other VMs hosted on the same platform. Intel® TDX also enhances defences against physical access attacks on platform memory, such as cold-boot attacks and DRAM interface intrusions. To achieve this high level of security, Intel® TDX incorporates new CPU security extensions that provide three essential security features:
 - **Memory Isolation through Main Memory Encryption:** CPUs equipped with confidential computing capabilities include an AES-128 hardware encryption engine within their memory controller. This engine encrypts and decrypts memory pages whenever there is a memory read or write operation. Instead of storing workload code and data in plain text in system memory, they are encrypted using a hardware-managed encryption key. This encryption and decryption process happens seamlessly within the CPU, ensuring strong memory isolation for confidential workloads.
 - **Additional CPU-Based Hardware Access Control Mechanisms:** CPUs with confidential computing capabilities introduce new instructions and data structures that allow auditing of security-sensitive tasks typically carried out by privileged system software. These tasks encompass memory management and access to platform devices. For

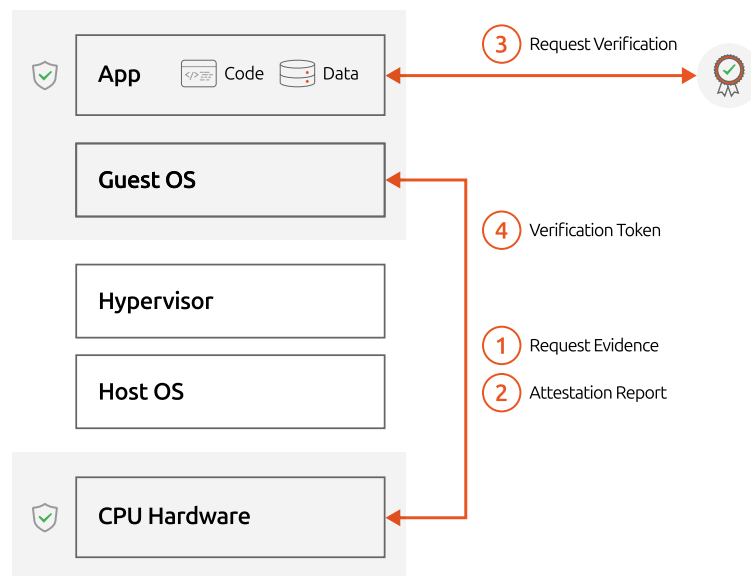


example, when reading memory pages mapped to confidential workloads, these new instructions also provide information about the last value written into the page. This feature helps prevent data corruption and replay attacks by detecting unauthorized modifications to memory pages.

- **Remote Attestation:** Enable a relying party (whether the owner of the workload or a user of the services provided by the workload) to confirm that the workload is operating on an Intel® TDX-enabled platform located within a TD before sharing data with it. Remote attestation allows both workload owners and consumers to digitally verify the version of the Trusted Computing Base (TCB) they are relying on to secure their data.
- **CPU-TEE:** Ubuntu confidential VMs that run on the AMD 4th Gen EPYC processors with SEV-SNP protect the workload's computation while in the CPU. It protects the computation through two major processes:
 - **Run-time confidentiality:** the DRAM of your Ubuntu CVMs is kept encrypted thanks to the new AES-128 hardware encryption engine that sits within the CPU memory controller. This engine encrypts and decrypts memory pages whenever there is a memory read or write operation. Instead of having workload code and data in plain text in system memory, they are encrypted using a hardware-managed encryption key. This encryption and decryption process happens seamlessly within the CPU, ensuring strong memory isolation for confidential workloads.
 - **Run-time Integrity:** Ubuntu CVMs make use of the new AMD SEV SNP instructions and data structures that allow auditing of security-sensitive tasks typically carried out by privileged system software. These tasks encompass memory management and access to platform devices. For example, when reading memory pages mapped to confidential workloads, these new instructions also provide information about the last value written into the page. This feature helps prevent data corruption and replay attacks by detecting unauthorized modifications to memory pages.
- **GPU-TEE:** NVIDIA H100 Tensor Core GPUs, which protect the confidentiality and integrity of the workload's computation within the GPU.
 - **Confidential AI on Azure** combines the strengths of AMD SEV-SNP technology with NVIDIA H100 GPUs. This architecture provides a fortified environment that secures data in use, in transit, and at rest. Ubuntu confidential VMs operate on AMD 4th Gen EPYC processors with SEV-SNP, which encrypts and integrity-protects the entire address space of the VM using hardware-managed keys. This prevents

unauthorized access or modification by any code outside the TEE. In addition, The NVIDIA H100 Tensor Core GPUs further extend these protections to the GPU, ensuring that all computations within the GPU are secure.

- Encrypted PCIe Communication: All traffic between the VM and GPU is encrypted and integrity-protected, safeguarding against advanced attacks.
- Attestation: This feature allows cryptographic verification of the security claims of both CPU and GPU TEEs, ensuring that data is processed only for its intended purpose.



By integrating these components into a cohesive solution, such as RAG deployments, confidential AI becomes feasible and practical, allowing organizations to harness the power of AI while maintaining the highest standards of data security and confidentiality. Confidential AI can then be further augmented with cryptographic primitives, such as differential privacy, which protect the workload from further sophisticated data leakage.

Confidential VM in Azure

Microsoft Azure has announced the [general availability](#) of their confidential virtual machines with NVIDIA H100 Tensor Core GPUs, powered by Ubuntu. This offering combines the hardware-based protection of AMD EPYC processors with NVIDIA's latest GPU technology to enable secure and high-performance AI workloads in the cloud. Similarly, Azure also announced that Intel® TDX on 4th Gen Intel® Xeon Scalable Processors [is available for public preview](#). Combining these technologies allows sensitive sectors to unlock AI adoption by addressing previous concerns of critical data privacy.

The objective is to deploy the RAG workflow outlined above in Microsoft® Azure Cloud’s confidential virtual machines. The recommended Azure instances listed below represent the minimum requirements for running three nodes of OpenSearch and one node of KServe.

Azure is one of the popular public clouds. By using Azure, users and builders of RAG can benefit from deploying scalable, high-performance, and secure RAG systems capable of handling complex NLP (Natural Language Processing) tasks. Azure offers network isolation for RAG workflows, data security through disk and storage encryption, and high-performance data processing with large network bandwidth. Additionally, it provides native monitoring and logging features, as well as compliance systems that include various certifications.

Application	Azure instances
Charmed OpenSearch	NVIDIA H100 Tensor Core GPU NCCads_H100_v5 series AMD 3rd Gen EPYC processors DCasv5 series ECasv5 and ECadsv5-series Intel® TDX on 4th Gen Xeon Scalable Processors DCesv5 series ECesv5 and ECedsv5-series
KServe	NVIDIA H100 Tensor Core GPU: NCCads_H100_v5 series

Deployment guide

In this guide, we will walk you through setting up a RAG pipeline. We will use Charmed OpenSearch for efficient search retrieval and KServe for machine learning inference.

PREREQUISITES

In order to follow along with and complete this guide, you will need:

- A working Azure subscription with necessary permissions to create VMs and networks, and to create Entra ID app registration.
- Any Ubuntu machine can use Juju CLI and operate the deployment later on. This could be a VM, laptop, or any other machine.
- Knowledge of the Juju ecosystem, OpenSearch, and KServe.

INSTALL JUJU AND CONFIGURE AZURE CREDENTIALS

At the time of writing this tutorial, the latest stable version of Juju is 3.5/stable, and we will be using it for OpenSearch bootstrap.

1. Install [Juju](#) (on the client machine).

```
$ sudo snap install juju
```

2. Perform CLI app registration in [Microsoft Entra ID](#), to allow Juju CLI create/modify and delete Azure resources (Azure CLI).

```
$ az ad app create --display-name jujucli
$ appid=$(az ad app list --display-name jujucli --query "[].{id: appId}" -o tsv)
$ az ad app credential list --id $appid --query keyId -o tsv
$ az ad app credential reset --id $appid
{
  "appId": "xxxxxx",
  "password": "qqqqq", "tenant": "zzzzz"
}
```

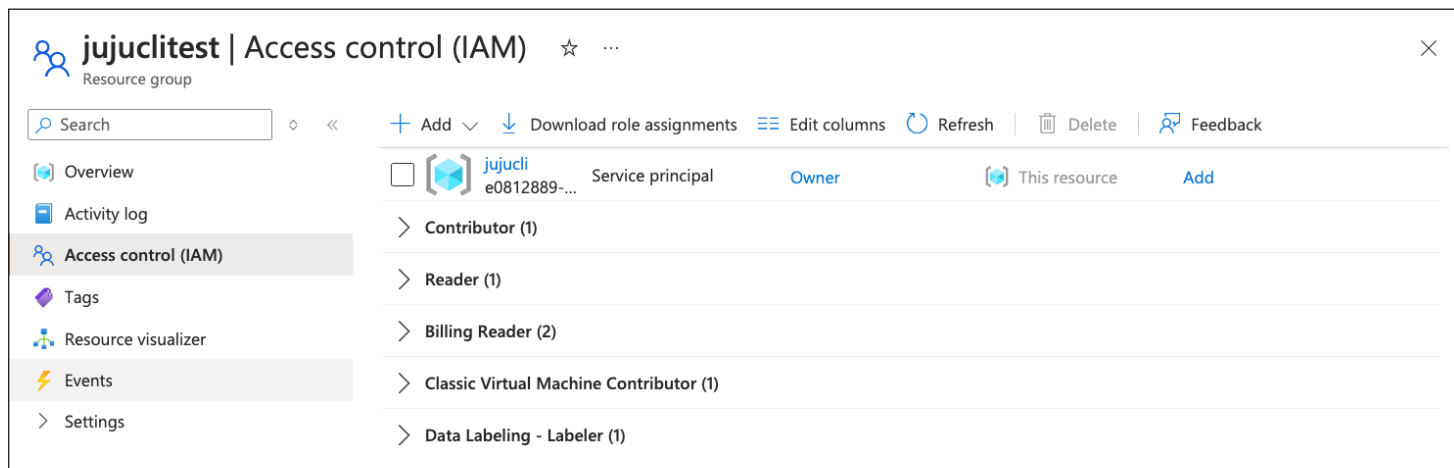
3. Create service principal (Azure CLI).

```
$ az ad sp create --id $appid
$ spObjectId=$(az ad sp show --id $appid --query id -o tsv)
```


- Grant this application Owner role on a resource we want to use, for example, resource group(1) of full subscription(2) (Azure CLI).

```
$ az role assignment create --assignee $spObjectId --role Owner --scope /subscriptions/xxxxxx/resourcegroups/RG
$ az role assignment create --assignee $spObjectId --role Owner --scope /subscriptions/xxxxxx
```

- Now inside our resource group (or subscription) Access Control IAM tab we should see this application as the owner.



- Create credentials.yaml file with the info we gathered in previous steps (on the client machine).

```
$ cat credentials.yaml
credentials:
  azure:
    azure_cloud:
      auth-type: service-principal-secret
      application-id: xxxxx
      application-password: qqqqq
      subscription-id: zzzzz
```

- Now add these credentials to Juju (on the client machine).

```
$ juju add-credential -f credentials.yaml --client azure
```

- Specify the Azure region you would like to deploy Charmed OpenSearch to (on the client machine).

```
$ juju default-region azure canadaeast
```


Bootstrap Juju controller and create Juju model for Charmed OpenSearch

1. Bootstrap the Juju controller to Azure (on the client machine). Deploy your resources to Resource group (RG) and VNET.

```
$ juju bootstrap --constraints allocate-public-ip=false --config resource-group-name=RG --config network=VNET azure
```

2. Add the model (on the client machine).

```
$ juju add-model opensearch --config resource-group-name=RG --config network=VNET
```

3. Add model-config. This config is required for Charmed OpenSearch (on the client machine).

```
$ cat <<EOF > cloudinit-userdata.yaml
cloudinit-userdata: |
  postruncmd:
    - echo "vm.max_map_count=262144" | tee -a /etc/sysctl.conf
    - echo "vm.swappiness=0" | tee -a /etc/sysctl.conf
    - echo "net.ipv4.tcp_retries2=5" | tee -a /etc/sysctl.conf
    - echo "fs.file-max=1048576" | tee -a /etc/sysctl.conf
    - sudo sysctl -p
EOF

$ juju model-config --file=./cloudinit-userdata.yaml
```

Deploy Charmed OpenSearch and set up the RAG services

1. Add appropriate machines to the juju model. We are adding 3 machines to deploy Charmed OpenSearch in a highly-available configuration (on the client machine).

```
$ juju add-machine --constraints="instance-type=Standard_DC4es_v5 allocate-public-ip=false zones=canadaeast" -n 3
```

2. Start Charmed OpenSearch deployment to client machines.

```
$ juju deploy opensearch --channel 2/edge --to 0,1,2
```

- Nodes in the Charmed OpenSearch cluster require TLS to communicate securely. Therefore, the OpenSearch charm will be blocked until TLS is configured. For this deployment we will use a `tls-certificates-operator` with self-signed certificate and relate it with OpenSearch (on the client machine).

```
$ juju deploy tls-certificates-operator
$ juju config tls-certificates-operator generate-self-signed-certificates="true" ca-common-name="Tutorial CA"
$ juju relate tls-certificates-operator opensearch
```

- After a few minutes, Charmed OpenSearch deployment should be in a ready/active state.

```
Model          Controller      Cloud/Region  Version  SLA          Timestamp
opensearch     azure-canadaeast azure/canadaeast 3.5.1    unsupported  15:01:57Z

App            Version  Status  Scale  Charm                    Channel
opensearch           active    3  opensearch                2/edge
tls-certificates-operator active    1  tls-certificates-operator latest/stable

Unit              Workload  Agent  Machine  Public address  Ports      Message
opensearch/0*      active   idle    0         10.0.0.7        9200/tcp
opensearch/1       active   idle    1         10.0.0.6        9200/tcp
opensearch/2       active   idle    2         10.0.0.8        9200/tcp
tls-certificates-operator/0* active   idle    3         52.242.22.73

Machine  State  Address      Inst id      Base           AZ  Message
0        started 10.0.0.7     juju-1b4e59-0 ubuntu@22.04
1        started 10.0.0.6     juju-1b4e59-1 ubuntu@22.04
2        started 10.0.0.8     juju-1b4e59-2 ubuntu@22.04
3        started 52.242.22.73 juju-1b4e59-3 ubuntu@22.04
```

- Get the admin user credentials (on the client machine).

```
$ juju run opensearch/leader get-password
...
password: os-password (saved)
username: admin

$ export OS_PASSWORD=
$ export OS_USERNAME=admin
$ export OS_ENDPOINT=https://10.0.0.7:9200
```

- Register a model group. Take note of the `model_group_id`.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/model_groups/_register" \
-H 'Content-Type: application/json' -d'
{
  "name": "rag-model-group",
  "description": "A model group for RAG use case models"
}'

{"model_group_id":"lw0H55EBiw8MvxrDTQdM","status":"CREATED"}
```

7. Register a sentence transformer model to the model group.

Choosing a transformer model varies with every case.

In most cases, starting with a general-purpose model for your desired modalities would be a sensible baseline.

We will choose an OpenSearch-provided pre-trained sentence transformer model, and take note of the vector dimensions.

For example: `huggingface/sentence-transformers/msmarco-distilbert-base-tas-b` with a 768-dimensional dense vector space.

```
-XPOST "$OS_ENDPOINT/_plugins/_ml/models/_register" \
-H 'Content-Type: application/json' -d'
{
  "name": "huggingface/sentence-transformers/msmarco-distilbert-base-tas-b",
  "version": "1.0.2",
  "model_group_id": "lW0H55EBiw8MvxrDTQdM",
  "model_format": "TORCH_SCRIPT"
}'
{"task_id":"lm0H55EBiw8MvxrDqwf5","status":"CREATED"}
```

8. Take note of the `task_id` for this task and check the status by calling the API.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XGET "$OS_ENDPOINT/_plugins/_ml/tasks/lm0H55EBiw8MvxrDqwf5"

{"model_id":"l20H55EBiw8MvxrDrwCL","task_type":"REGISTER_MODEL","function_name":"TEXT_
EMBEDDING","state":"COMPLETED","worker_node":["hy5M6dC9SqW2HR0QCRt5lw"],"create_time":1726157925367,"last_update_
time":1726157967875,"is_async":true}
```

9. Take note of the `model_id` and deploy the sentence transformer model.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/models/l20H55EBiw8MvxrDrwCL/_deploy"

{"task_id":"mG0I55EBiw8MvxrDLAeV","task_type":"DEPLOY_MODEL","status":"CREATED"}
```

10. To test the deployed model, you can use the “predict” API of the deployed model. It will return a sentence embedding vector with the sentence transformer model dimensionality.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/_predict/text_embedding/l20H55EBiw8MvxrDrwCL" \
-H 'Content-Type: application/json' -d'
{
  "text_docs":["What is Ubuntu Pro?"],
  "return_number": true,
  "target_response": ["sentence_embedding"]
}
```

```
}'

{"inference_results":[{"output":[{"name":"sentence_embedding", "data_type":"FLOAT32","shape":[768],"data": [ . . . ]}]]}]}
```

11. Create an ingest pipeline using the sentence transformer model.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPUT "$OS_ENDPOINT/_ingest/pipeline/rag-ingest-pipeline" \
-H 'Content-Type: application/json' -d'
{
  "description": "An RAG ingest pipeline",
  "processors": [
    {
      "text_embedding": {
        "model_id": "l20H55EBiw8MvxrDrwCL",
        "field_map": {
          "text": "sentence_embedding"
        }
      }
    }
  ]
}'

{"acknowledged":true}
```

12. Create a kNN index with the ingestion pipeline. Please use the [upstream documentation about k-NN indexes](#) to find more information about the engine tuning, e.g., space types and encoder parameters.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPUT "$OS_ENDPOINT/knn-faiss-avx2-index1" \
-H 'Content-Type: application/json' -d'
{
  "settings": {
    "index.knn": true,
    "default_pipeline": "rag-ingest-pipeline"
  },
  "mappings": {
    "properties": {
      "id": {
        "type": "text"
      },
      "sentence_embedding": {
        "type": "knn_vector",
        "dimension": 768,
        "method": {
          "name": "hnsw",
          "engine": "faiss",
          "space_type": "l2",
          "parameters": {
            "encoder": {
              "name": "sq",
              "parameters": {
                "type": "fp16"
              }
            }
          }
        }
      }
    }
  },
}
```

```

        "ef_construction": 256,
        "m": 8
      }
    },
    "text": {
      "type": "text"
    }
  }
}
}'

{"acknowledged":true,"shards_acknowledged":true,"index":"knn-faiss-avx2-index1"}

```

13. Load documents to the index that can be later used as context for the augmented generation.

```

$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
  -XPOST "$OS_ENDPOINT/knn-faiss-avx2-index1/_doc" \
  -H 'Content-Type: application/json' -d'
{
  "text": "..."
}'

```

14. Test the Neural Search.

```

$ curl --insecure -u '$OS_USERNAME:$OS_PASSWORD' \
  -XGET "$OS_ENDPOINT/knn-faiss-avx2-index1/_search" \
  -H 'Content-Type: application/json' -d'
{
  "_source": {
    "excludes": [
      "sentence_embedding"
    ]
  },
  "query": {
    "neural": {
      "sentence_embedding": {
        "query_text": "What is Ubuntu Pro?",
        "model_id": "l20H55EBiw8MvxrDrwCL",
        "k": 15
      }
    }
  }
}'

{"took":1044,"timed_out":false,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0},"hits":{"total":{"value":5,"relation":"eq"},"max_score":0.024580367,"hits":[ . . . ]}}

```

15. Enable private addresses, add the LLM endpoint to the trusted connector, and create a model connector to your LLM."

Check our article on how to deploy your own LLM, for example Llama2, on Kubernetes using Kserve and VLLM.

For the purposes of this guide, we assume that the LLM is already hosted and accessible via the API endpoint:

`llama-2-13b-predictor.vllm.10.6.0.13.nip.io.`

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPUT "$OS_ENDPOINT/_cluster/settings" \
-H 'Content-Type: application/json' -d'
{
  "persistent": {
    "plugins.ml_commons.connector.private_ip_enabled": true,
    "plugins.ml_commons.trusted_connector_endpoints_regex": [
      "^http://.*[a-z0-9-]\\\\.10\\.\\.6\\.\\.0\\.\\..*[a-z0-9-]\\\\.nip\\.\\.io/.*$"
    ]
  }
}'

$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/connectors/_create" \
-H 'Content-Type: application/json' -d'
{
  "name": "vLLM Connector",
  "description": "OpenAI-compatible vLLM API",
  "version": 2,
  "protocol": "http",
  "parameters": {
    "endpoint": "llama-2-13b-predictor.vllm.10.6.0.13.nip.io",
    "model": "TheBloke/Llama-2-13B-chat-GPTQ",
    "temperature": 0.03,
    "repetition_penalty": 1.13
  },
  "actions": [
    {
      "action_type": "predict",
      "method": "POST",
      "url": "http://${parameters.endpoint}/v1/chat/completions",
      "request_body": "{ \"model\": \"${parameters.model}\", \"messages\": ${parameters.prompt}, \"temperature\": ${parameters.temperature}, \"repetition_penalty\": ${parameters.repetition_penalty} }"
    }
  ]
}'

{"connector_id": "kG0A55EBiw8MvxrDgQeD"}
```

16. Register the LLM connector.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/models/_register" \
-H 'Content-Type: application/json' -d'
{
  "name": "KServe Inference Service: Llama-2-13B-chat",
  "function_name": "remote",
  "description": "RAG LLM test model",
  "connector_id": "kG0A55EBiw8MvxrDgQeD"
}'

{"task_id": "km0C55EBiw8MvxrDVgeF", "status": "CREATED", "model_id": "k20C55EBiw8MvxrDWQfl"}
```

17. Get the LLM connector model id using the task id.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XGET "$OS_ENDPOINT/_plugins/_ml/tasks/km0C55EBiw8MvxrDVgeF"

{"model_id": "k20C55EBiw8MvxrDWQfl", "task_type": "REGISTER_MODEL", "function_name": "REMOTE", "state": "COMPLETED", "worker_node": ["hy5M6dC9SqW2HR0QCRt5lw"], "create_time": 1726157575317, "last_update_time": 1726157577102, "is_async": false}
```

18. Deploy the LLM connector model.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/models/k20C55EBiw8MvxrDWQfl/_deploy"
{"task_id": "lg0D55EBiw8MvxrDKQcd", "task_type": "DEPLOY_MODEL", "status": "COMPLETED"}
```

19. Create an agent for the conversational flow.

```
$ curl '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/agents/_register" \
-H 'Content-Type: application/json' -d'
{
  "name": "test-agent-vector-mlmodel-rag",
  "type": "conversational_flow",
  "description": "This is a demo agent for conversational flow",
  "app_type": "rag",
  "memory": {
    "type": "conversation_index"
  },
  "tools": [
    {
      "type": "VectorDBTool",
      "parameters": {
        "model_id": "l20H55EBiw8MvxrDrwCL",
        "index": "knn-faiss-avx2-index1",
        "embedding_field": "sentence_embedding",
        "source_field": ["text"],
        "input": "${parameters.question}",
        "doc_size": 10,
        "k": 30
      }
    },
    {
      "type": "MLModelTool",
      "description": "A RAG LLM tool to answer questions",
      "parameters": {
        "model_id": "k20C55EBiw8MvxrDWQfl",
        "prompt": "[ { \"role\": \"Context:\", \"content\": \"${parameters.VectorDBTool.output}\" }, { \"role\": \"Chat history:\", \"content\": \"${parameters.chat_history:-}\" }, { \"role\": \"User instruction:\", \"content\": \"${parameters.question}\" } ]"
      }
    }
  ]
}'

{"agent_id": "ym066JEBiw8MvxrDJwgr"}
```


Ask and start conversational flow with your RAG

1. Finally, ask a question to start a new conversation.

```
$ curl -u '$OS_USERNAME:$OS_PASSWORD' \
-XPOST "$OS_ENDPOINT/_plugins/_ml/agents/ym066JEBiw8MvxrDJwgR/_execute" \
-H 'Content-Type: application/json' -d'
{
  "parameters": {
    "question": "What is the best Linux distribution?"
  }
}'

{"inference_results": [{"output": [{"name": "memory_id", "result": "41mILZEBt5crDBEqzptK"}, {"name": "parent_message_id", "result": "5FmILZEBt5crDBEq0puo"}, {"name": "MLModelTool", "result": "{\n  \"id\": \"chatcml-9tdfAtRvy2ZIXI3v32vbofXFIoV1y\", \"object\": \"chat.completion\", \"created\": 1.723047532E9, \"model\": \"TheBloke/Llama-2-13B-chat-GPTQ\", \"choices\": [{\n    \"index\": 0.0, \"message\": {\n      \"role\": \"assistant\", \"content\": \" . . . \"\n    }, \"finish_reason\": \"stop\"}], \"usage\": {\n      \"prompt_tokens\": 361.0, \"completion_tokens\": 92.0, \"total_tokens\": 453.0\n    }\n  }"}]}
```

2. Next, use the memory id to continue the conversation.

```
$ curl -u $OS_USERNAME:$OS_PASSWORD \
-XPOST "$OS_ENDPOINT/_plugins/_ml/agents/ym066JEBiw8MvxrDJwgR/_execute" \
-H 'Content-Type: application/json' -d'
{
  "parameters": {
    "question": "Which one can be used for confidential VMs in Azure Cloud?",
    "memory_id": "41mILZEBt5crDBEqzptK",
    "message_history_limit": 3
  }
}'

{"inference_results": [ . . . ] }
```

Conclusion

As we have demonstrated, RAG offers numerous benefits and has become a powerful technique for enhancing generative AI applications. Open source tools like OpenSearch and KServe have been critical in driving innovation within the RAG ecosystem. Our guide showcases their utility in enabling RAG for your AI tools.

Choosing the right tool is essential for optimizing your RAG system, as is selecting the appropriate cloud platform and hardware. Additionally, ensuring the confidentiality and security of data and machine learning models is vital in RAG systems. Confidential AI addresses this issue of data and machine learning models' confidentiality by providing a hardware-rooted execution environment that spans both the CPU and GPU. This environment enhances the protection of AI data and code at runtime, safeguarding it against privileged system software – such as the hypervisor or host operating system – and privileged operators in the cloud.

Before launching your project – whether for a proof of concept, development, testing, or full production – careful planning and consideration of your AI infrastructure are crucial. Making informed decisions about the overall use case, business needs, technology tools, cloud environments, and hardware will set the foundation for a successful and scalable RAG solution. For our use case, Azure offers a robust environment, while accelerators like Intel, AMD, and NVIDIA help to scale your RAG services to meet varying security, latency, throughput, and computing requirements. This guide showed how to build a RAG system using open source tools and the benefits of deploying it in a confidential virtual machine. Understanding these fundamental concepts is crucial for developing solutions tailored to your specific use case. After following this guide, you should be able to deploy and configure various open source tools to create an AI chatbot.



Confidential RAG in Canonical

BUILD THE RIGHT RAG ARCHITECTURE AND APPLICATION WITH THE CANONICAL RAG WORKSHOP

Canonical offers a 5-day workshop designed to help you start building your enterprise RAG systems. By the end of the workshop, you will have a thorough understanding of RAG and LLM theory, architecture, and best practices. Together, we will develop and deploy solutions tailored to your specific needs. You can learn more about the workshop's goals and outcomes by reading our workshop datasheet.

[Download the datasheet here.](#)

LEARN AND USE BEST-IN-CLASS RAG TOOLING ON ANY HARDWARE AND CLOUD

Unlock the benefits of RAG with open source tools designed for your entire data and machine learning lifecycle. Run RAG-enabled large language models on any hardware and cloud platform, whether in production or at scale.

Canonical offers [enterprise-ready AI infrastructure](#) along with open source [data](#) and [AI](#) tools to help you kickstart your RAG projects.

SECURE YOUR AI STACK WITH CONFIDENCE

Enhance the security of your GenAI projects while mastering best practices for managing your software stack. Discover ways to safeguard your code, data, and machine learning models in production with [Confidential AI](#).

Send us a message to learn more about confidential RAG

[Get in touch >](#)

AUTHORS

Michelle Tabirao
Alex Litvinov
Juan Pablo Noreña

Ubuntu, Kubuntu, Canonical and their associated logos are the registered trademarks of Canonical Ltd.	All other trademarks are the properties of their respective owners.	Any information referred to in this document may change without notice and Canonical will not be held responsible for any such changes.
---	---	---

Canonical Limited

Registered in Isle of Man,
Company number 110334C

Registered Office

2nd Floor Clarendon House
Victoria Street, Douglas
IM1 2LN
Isle of Man

